Record - Week 04

Topic: MongoDB and Mongoose

Name: _____ Roll No: _____ Date: _____

Aim

To connect Node.js to MongoDB and perform CRUD operations on a User collection using a Mongoose model and controller.

Software Requirements

Node.js 18+, MongoDB Community Server, MongoDB Compass, Mongoose, VS Code

Procedure

1 Start MongoDB locally and install mongoose.

2 Create dbConfig.js that connects using MONGO_URI.

3 Define the User schema in models/User.js.

4 Write the five CRUD handlers in controllers/userController.js.

5 Run the operations and verify the documents in MongoDB Compass.

Program

dbConfig.js

```
const mongoose = require("mongoose");

const MONGO_URI = process.env.MONGO_URI || "mongodb://127.0.0.1:27017/expense_tracker";

async function connectDB() {
  try {
    await mongoose.connect(MONGO_URI);
    console.log("MongoDB connected");
  } catch (err) {
    console.error("MongoDB connection failed:", err.message);
    process.exit(1);
  }
}

module.exports = connectDB;
```

models/User.js

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true },
    monthlyBudget: { type: Number, min: 0, default: 0 },
    currency: { type: String, enum: ["INR", "USD", "EUR"], default: "INR" },
  },
  { timestamps: true }
);

module.exports = mongoose.model("User", userSchema);
```

controllers/userController.js

```
const User = require("../models/User");

// CREATE
exports.createUser = async (req, res) => {
  try {
    const user = await User.create(req.body);
    res.status(201).json(user);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
};

// READ
exports.getUsers = async (req, res) => {
  const users = await User.find().sort({ createdAt: -1 });
  res.json(users);
};

exports.getUser = async (req, res) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ error: "User not found" });
    res.json(user);
  } catch (err) {
    res.status(400).json({ error: "Invalid user id" });
  }
};

// UPDATE
exports.updateUser = async (req, res) => {
  try {
    const user = await User.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true,
    });
    if (!user) return res.status(404).json({ error: "User not found" });
    res.json(user);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
};

// DELETE
exports.deleteUser = async (req, res) => {
  try {
    const user = await User.findByIdAndDelete(req.params.id);
    if (!user) return res.status(404).json({ error: "User not found" });
    res.json({ message: "User deleted" });
  } catch (err) {
    res.status(400).json({ error: "Invalid user id" });
  }
};
```

Output

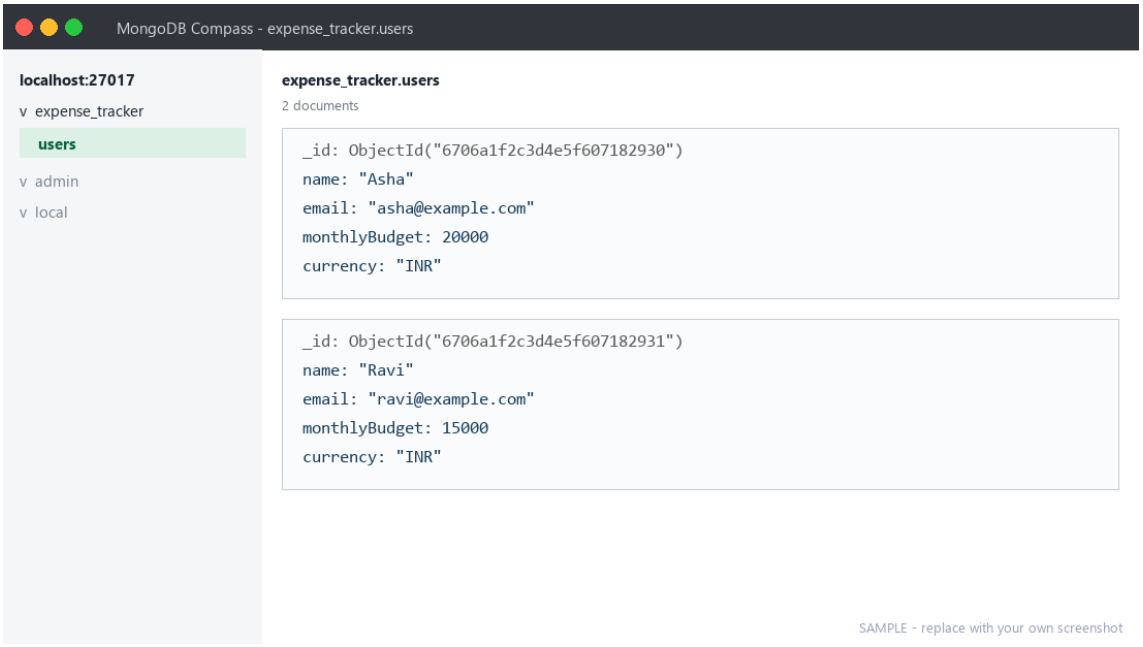


Fig: mongodb compass db

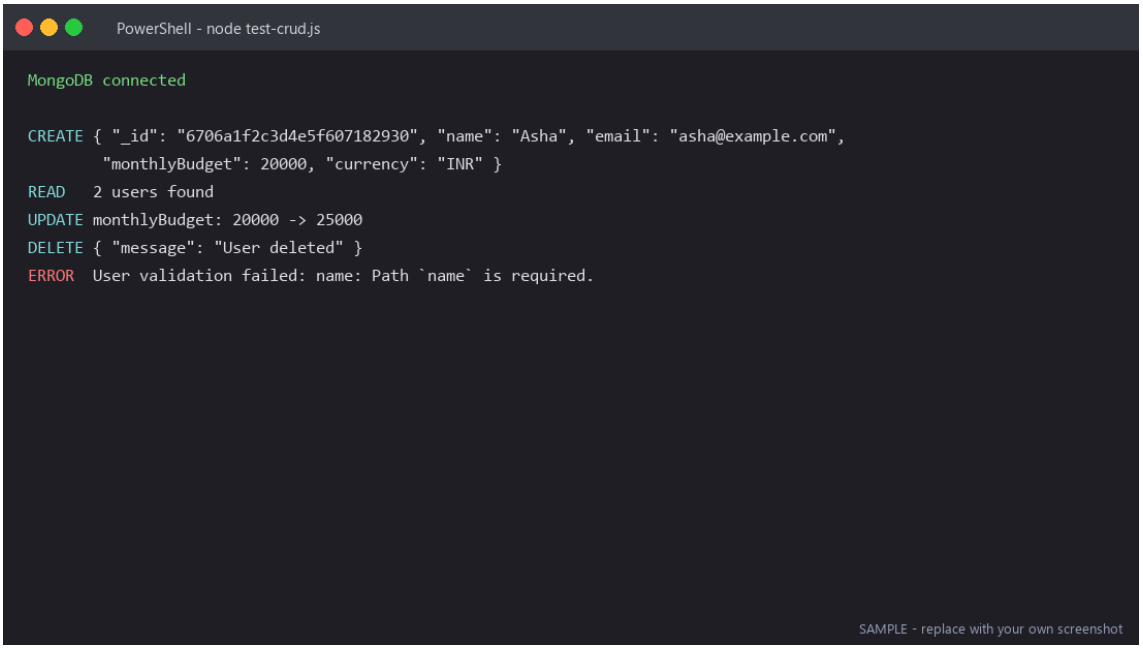


Fig: crud operations output

Result

The program was executed successfully and the expected output was obtained.